BLG 411E Software Engineering Term Project

Project: Witch Puzzles

Group: Witch

Members

Ertuğrul Şentürk

Utku Biçer

Hüseyin Şimşek

Oğuzhan Altıntaş

Oğuz Eren Kacar

Lucas Holczinger

Ata Türköz

05.01.2025

Test Specification Document

Introduction

Goal

The goal of our testing is to ensure that our Puzzles website functions correctly, reliably, and securely, meeting both user expectations and project requirements. Through testing, we aim to identify and eliminate potential issues in functionality and usability before deploying the system to users.

Our testing strategy includes both black-box testing and white-box testing. Black-box testing focuses on validating the system's behavior by testing inputs and outputs against expected results without considering the internal code structure. White-box testing, on the other hand, involves examining the internal logic, structure, and individual components of the code to verify their correctness and robustness.

By adopting a systematic and comprehensive testing process, we aim to deliver a high-quality product that is reliable under all expected conditions.

Contents and Organization

The document is structured as follows: - Test Plan: Outline the overall strategy, subjects and methods used for testing - Test Results: Results and analysis of information gathered from our tests

Test Plan

Testing Strategy

Unit Testing

We use pytest for unit testing, focusing on individual components like functions, methods, and classes to ensure they behave correctly. Unit tests are written for several different modules to check logic, edge cases, and proper error handling.

Integration Testing

We follow a bottom-up approach to integration testing, starting with lower-level services and building upwards. This ensures smooth interaction between modules, such as the correct flow of data between backend services and the database, and proper API communication.

System Testing

For system testing, we use both the black box and white box testing methods. We validate that the website produces correct outputs based on specified inputs and handles edge cases appropriately.

Test Subjects

Frontend Testing Subjects

- Ensure that the landing page loads correctly and is responsive on various devices.
- Validate that users can log in using their email addresses and using Google and appropriate error messages are shown for invalid credentials.
- Test the interface for selecting puzzles, displaying puzzle data, and interacting with the solution input.
- Ensure the system accurately accepts or rejects solutions, and provides appropriate feedback.
- Test that leaderboard data is displayed correctly after updates to the table.
- Ensure users can view their profile details.

Backend Testing Subjects

- Ensure that the user registration, login, and authentication flow works correctly with the Firebase Authentication service.
- Validate the correct storage of user data after successful registration (e.g., name, email, password hash).
- Check that invalid credentials are properly rejected.
- Test user data retrieval to ensure correct information is returned.
- Ensure that generating, solving and cathegorizing puzzles generates proper results and does not cause high stress on the machine.
- Ensure puzzle data is correctly stored for each puzzle after populating the database.
- Ensure that puzzles are fetched correctly based on user selection (e.g., easy, medium, hard).
- Ensure that puzzle completion recordfs are updated correctly in the database.
- Validate that the leaderboard correctly updates when a new puzzle is solved and that rankings are recalculated based on new completion times.

Black Box Testing

Description	Test Input	Expected Output	Mapped Use Case
Test User Registration with valid data	Valid name, email, password	Registration successful	User Registration
Test User Registration with invalid email format	Valid name, invalid email, valid password	Invalid email address error	User Registration
Test User Registration with missing password	Valid name, valid email, empty password	Missing password field error	User Registration
Test User Sign in using Google	None	Pop up for sign in	User Registration
Test Random Puzzle Fetching	None	Puzzle information	Puzzle Solving
Test Puzzle Submission with correct solution	Correct puzzle solution	Success message	Puzzle Solving
Test Puzzle Submission with incorrect solution	Incorrect puzzle solution	Incorrect solution error	Puzzle Solving
Test Leaderboard retrieval in a time range	Difficulty level, time interval	Leaderboard information	Viewing the Leaderboard

White Box Testing

White box testing requires us to understand the structure of the source code (including modules, functions, and classes) and list the corresponding unit tests, mocks, and tools.

Backend Source Code Structure

```
Modules:
        User Service: Handles user saving to database, user
profile updates and syncing with Firebase.
        Sudoku Service: Manages puzzle data fetching and
validation.
        Sudoku Registry Service: Manages solved sudoku records
and leaderboard data fetching.
        Email Util: Helper for sending emails to users
    Database: Manages storage and retrieval of user data,
puzzles, and leaderboard entries.
Functions:
    SudokuService:
        get_random_sudoku_by_difficulty(): Gets a random sudoku
by difficulty from the database
        get_sudoku_by_id(): Gets a sudoku by id from the database
        populate_sudoku_registry(): Populates the sudoku database
        validate_sudoku(): Validates if a sudoku is solved
    SudokuRegsitryService:
        get_leaderboard_today(): Gets today's leaderboard
        get_leaderboard_week(): Gets week's leaderboard
        get_leaderboard_month(): Gets month's leaderboard
        get_leaderboard_all_time(): Gets all time's leaderboard
        get_leaderboard(): Gets a leaderboard by difficulty and
optional beginning time
    UserService:
        getUserByFirebaseId(): Gets user by associated Firebase
id
        createUser(): Creates user with associated Firebase id in
the database
        updateUser(): Updates user with associated Firebase id in
the database
        am_i_admin(): Checks if user with associated Firebase id
is an admin
```

Required Mocks

Database Mocks: For simulating puzzle data retrieval during tests.

Testing Tools

pytest & pytest-cov: For unit testing individual functions and modules. And coverage reporting.

Test Results

Analysis

Coverage report generated by pytest module (with the pytest-cov plugin).

```
tests/test_main.py .                                                     [  2%]
tests/test_sudoku_grid.py .............                                  [ 28%]
tests/test_sudoku_registry_repository.py ....FFF                         [ 42%]
tests/test_sudoku_registry_service.py F..F                               [ 51%]
tests/test_sudoku_repository.py .....                                    [ 61%]
tests/test_sudoku_service.py EFEEE                                       [ 71%]
tests/test_user_repository.py ......                                     [ 83%]
---------- coverage: platform linux, python 3.12.8-final-0 -----------

========================================== short test summary info ==========================================
FAILED tests/test_sudoku_registry_repository.py::test_get_leaderboard - AssertionError: assert <MagicMock
 na...400088248016'> == [<app.entitie...786acc3c56a0>]
FAILED tests/test_sudoku_registry_repository.py::test_get_user_place_in_leaderboard - assert None is not
None
FAILED tests/test_sudoku_registry_repository.py::test_get_user_records - assert 0 == 1
FAILED tests/test_sudoku_registry_service.py::test_get_leaderboard_today - AssertionError: assert 3 == 1
FAILED tests/test_sudoku_registry_service.py::test_submit_sudoku_incorrect_solution - AssertionError: ass
ert not True
FAILED tests/test_sudoku_service.py::test_get_sudoku_by_id - TypeError: SudokuService.__init__() missing
1 required positional argument: 'user_service'
ERROR tests/test_sudoku_service.py::test_get_random_sudoku_by_difficulty
ERROR tests/test_sudoku_service.py::test_validate_sudoku_valid_solution
ERROR tests/test_sudoku_service.py::test_validate_sudoku_invalid_solution
ERROR tests/test_sudoku_service.py::test_validate_sudoku_with_exception
============================== 6 failed, 39 passed, 1 warning, 4 errors in 20.19s ==============================
```

Logs

Unit test logs generated by the pytest module.

```
tests\test_sudoku_grid.py .............                                  [100%]

============================================================== 13 passed in 3.44s ==============================================================
```

```
tests\test_sudoku_registry_repository.py ....                            [100%]

============================================================== warnings summary ================================================================
app\core\database.py:14
  C:\visionbase\backend-puzzles\app\core\database.py:14: MovedIn20Warning: The ``declarative_base()`` function is now available as sqlalchemy.orm.declarative_bas
e(). (deprecated since: 2.0) (Background on SQLAlchemy 2.0 at: https://sqlalche.me/e/b8d9)
    Base = declarative_base()

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
============================================================ 4 passed, 1 warning in 0.71s ============================================================
```

```
tests\test_sudoku_registry_service.py ..                                 [100%]

============================================================== warnings summary ================================================================
app\core\database.py:14
  C:\visionbase\backend-puzzles\app\core\database.py:14: MovedIn20Warning: The ``declarative_base()`` function is now available as sqlalchemy.orm.declarative_bas
e(). (deprecated since: 2.0) (Background on SQLAlchemy 2.0 at: https://sqlalche.me/e/b8d9)
    Base = declarative_base()

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
============================================================ 2 passed, 1 warning in 1.10s ============================================================
```

```
tests\test_sudoku_repository.py .....                                    [100%]

============================================================== warnings summary ================================================================
app\core\database.py:14
  C:\visionbase\backend-puzzles\app\core\database.py:14: MovedIn20Warning: The ``declarative_base()`` function is now available as sqlalchemy.orm.declarative_bas
e(). (deprecated since: 2.0) (Background on SQLAlchemy 2.0 at: https://sqlalche.me/e/b8d9)
    Base = declarative_base()

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
============================================================ 5 passed, 1 warning in 1.09s ============================================================
```

```
tests\test_sudoku_service.py .......                                     [100%]

============================================================== warnings summary ================================================================
app\core\database.py:14
  C:\visionbase\backend-puzzles\app\core\database.py:14: MovedIn20Warning: The ``declarative_base()`` function is now available as sqlalchemy.orm.declarative_bas
e(). (deprecated since: 2.0) (Background on SQLAlchemy 2.0 at: https://sqlalche.me/e/b8d9)
    Base = declarative_base()

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
============================================================ 8 passed, 1 warning in 0.89s ============================================================
```

```
tests\test_user_repository.py ......                                                                                [100%]

============================================================= warnings summary ==============================================================
app\core\database.py:14
  C:\visionbase\backend-puzzles\app\core\database.py:14: MovedIn20Warning: The ``declarative_base()`` function is now available as sqlalchemy.orm.declarative_bas
e(). (deprecated since: 2.0) (Background on SQLAlchemy 2.0 at: https://sqlalche.me/e/b8d9)
    Base = declarative_base()

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
============================================================= 6 passed, 1 warning in 0.72s ==============================================================
```

```
tests\test_user_service.py ........                                                                                 [100%]

============================================================= warnings summary ==============================================================
app\core\database.py:14
  C:\visionbase\backend-puzzles\app\core\database.py:14: MovedIn20Warning: The ``declarative_base()`` function is now available as sqlalchemy.orm.declarative_bas
e(). (deprecated since: 2.0) (Background on SQLAlchemy 2.0 at: https://sqlalche.me/e/b8d9)
    Base = declarative_base()

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
============================================================= 8 passed, 1 warning in 0.69s ==============================================================
```